Regular Paper

A Reference Architecture for Mobile SOA

Behrouz Sefid-Dashti[1,*] and Jafar Habibi[2]

[1] *Department of Computer Engineering, Tehran North Branch, Islamic Azad University, Tehran, Iran*
[2] *Department of Computer Engineering, Sharif University of Technology, Tehran, Iran*

Received 3 July 2013; Revised 3 July 2013; Accepted 10 September 2013, after one or more revisions
Published online 22 April 2014 in Wiley Online Library (wileyonlinelibrary.com).
DOI 10.1111/sys.21279

ABSTRACT

Integration of mobile devices into a service-oriented system involves the consideration of architectural design principles and trade-offs among interdependent design decisions to cope with limitations of mobile devices in terms of resources available within a device and aspects of network connectivity. Mobile devices consume some services and provide others. A reference architecture (RA) that provides solutions, baseline and future insights is the priority in designing efficient mobile service oriented architecture (SOA). This paper discusses nuances of the concept of RA. The paper presents a survey of 26 mobile SOA patterns and proposes mobile SOA RA that extends Open Group SOA RA based on the catalogue of mobile SOA patterns proposed in this paper. Evaluations are made and discussed for both RA and its possible instantiation.
 Syst Eng 17: 407–425, 2014

Key words: mobile SOA reference architecture; reference architecture; mobile SOA pattern catalogue

1. INTRODUCTION

Service oriented architecture (SOA) is receiving attention relating to its potential application in business because of its considerable flexibility in aligning IT functions with business processes. There are several challenges that need to be overcome to create quality solutions using (mobile) services [Oliveira et al., 2010] and this has led to numerous reference models and different models of reference architecture (RA). A study by Oliveira et al. [2010], presents an analysis of more than 20 of these. The spread of smart phones has activated the mobile service market [Chujo et al., 2013]. Technologies such as smart phones and tablet devices enable the use and composition of services in mobile environments and these mobile devices are powerful enough to work as service providers; in addition there are service provision platform(s) and workflow engine(s) that support service provision and orchestration via legacy (yesterday) cell phones and PDAs which have less resources then smart phones and tablets [Plebani et al., 2012]; thus paving the way for "mobile SOA." Mobile phone consumers are increasingly using their devices for both personal and business use. Many employees can spend up to a third of their working day away from an allocated workstation, so mobile phones are increasingly being relied on as a way to address work from unspecific and various locations. Mobile phones can be used by employees to reach business decisions while away from their desks, to communicate and to access enterprise systems such as Enterprise Resource Planning, Customer Relationship Management, and Business Intelligence [Natchetoi, Kaufman, and Shapiro, 2008]. In 2008, there were approximately 3.3 billion connected mobile devices in the world [Natchetoi, Kaufman, and Shapiro, 2008]. A report in Strategy Analytics in 2012 estimated that there were over 1 billion smart phones in current use [IBM, 2012]. Gartner [2012] predicts that by 2015 over 80% of the handsets sold in mature markets will be smart phones, so that "Mobile Applications and Media Tablets" ranked second in Gartner's

*Author to whom all correspondence should be addressed (e-mail: bsefid-dashti@rasanegar.com).

2011 list of top strategic technologies [Gartner, 2010] and "Media Tablets and Beyond" ranked first in Gartner's list of top strategic technologies in 2012 [Gartner, 2011] to "Mobile Device Battles" and "Mobile Applications and HTML5" that ranked first in Gartner's 2013 list of top strategic technology trends [Gartner, 2012]. Each device is available on the market within a year of its introduction [Gurp, Karhinen, and Bosch, 2006]; hence developing mobile software in a device specific manner can serve to shrink its potential market in terms of time and size. Platform independent principles of SOA make SOA appropriate for integrating numerous different platforms associated with mobile and nonmobile devices. Loosely coupling and fractal natural of SOA match mobile device connection intermittence and scope changes, respectively. Integrating mobile devices into service-oriented systems with adherence to a Service Level Agreement requires coping with new challenges imposed by incorporating nonfunctional requirements (NFRs), which demands an architectural template otherwise known as a RA. No known RA for mobile SOAs has yet been developed. Pervasive computing and mobile SOAs have some attributes in common but sensors and ubiquitous control devices distinguish infrastructure and RA of pervasive computing [Liu and Li, 2006] and mobile SOAs.

This paper proposes a 10-layered RA for mobile services based on mobile SOAs from 2005 to 2013. This proposed RA distributes extracted mobile SOA patterns among layers of the Open Group SOA RA [Open Group, 2011] and introduces a new domain specific layer. The resulting RA is more concrete than its predecessor so that the abilities of Open Group SOA RA such as reacting to changes in infrastructure in order to streamline a service [Open Group, 2011], can be realized by several mobile SOA patterns of mobile SOA RA. This ability (being able to react to changes in infrastructure in order to streamline a service) is defined in the Quality of Service layer of Open Group SOA RA [Open Group, 2011]. It is realized by applying patterns corresponding to this ability, to the architectural design process of concrete mobile SOA solutions.

The remainder of this paper is structured as follows: Section 2. discusses definitions of RA. Section 3. considers requirements and architectural drivers of mobile SOAs. Section 4. explains extracted patterns and elicits architectural trends in mobile SOAs. Subsequent sections describe the proposed RA and an evaluation of results. Section 7. presents a summary and conclusions.

2. THE CONCEPT OF RA

The ISO/IEC/IEEE standard 42010:2011(E) defines (system) architecture as follows:

Definition 1. *"Fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution" [ISO/IEC/IEEE 42010, 2011].*

An accepted definition of software architecture [Bass, Clements, and Kazman, 2003] emphasizes the external visible properties of elements and relationships between those elements. Taylor, Medvidovic, and Dashofy [2009] defines it as "set of principal design decisions" and "the blueprint for a software system's construction and evolution" [Taylor, Medvidovic, and Dashofy, 2009].

The terms "system architecture" and "software architecture" are interchangeable in the context of RA, however they are applied quite differently [Bass, Clements, and Kazman, 2003]. System architecture is concentrated on the system of interest in relation to its environment and is associated with its evolution [Muller and Laar, 2011]. Product line architecture aims to develop a family of applications in a low-cost configurable manner. In contrast to system architecture, Architecting methods and architecture frameworks provide generic guidance for designing a broad set of systems with no knowledge of the particular service being addressed, be it insurance, banking, or air traffic control [Muller and Laar, 2011]. Thus architecture frameworks and architecting methods are not necessarily domain specific [Muller and Laar, 2011].

However RA is a domain specific concept [Muller and Laar, 2011] that forms a base to assist in developing new system architectures [Cloutier et al., 2010]. The definition of RA has recently attracted the attention of research committees [Muller and Laar, 2011; Cloutier et al., 2010]. The following listed attributes are some commonly used attributes used to describe RA:

- It is abstract and provides a base for system architecture [Bass, Clements, and Kazman, 2003; Muller and Laar, 2011; Nakagawa, Becker, and Maldonado, 2012; Taylor, Medvidovic, and Dashofy, 2009].
- It either is domain specific or stems from vision and strategy of an organization [Cloutier et al., 2010; Muller and Laar, 2011; Nakagawa, Becker, and Maldonado, 2012; Reed, 2002; Taylor, Medvidovic, and Dashofy, 2009].
- It mines the implicit and explicit knowledge of available (relevant) architectures for use in future architectures [Cloutier et al., 2010; Muller and Laar, 2011; Nakagawa, Becker, and Maldonado, 2012].
- It captures domain specific knowledge, commonly in the form of patterns [Cloutier et al., 2010; Muller and Laar, 2011; Reed, 2002]. RA can contain architectural patterns [Bass, Clements, and Kazman, 2003; Nakagawa, Becker, and Maldonado, 2012] that can be domain specific [Muller and Laar, 2011] or domain-independent [Angelov, Grefen, and Greefhorst, 2012].
- It captures common attributes of systems in a domain while supporting variability so that it can be applied to a specific system within a domain [Nakagawa, 2012]. Points of variations are explicitly defined [Taylor, Medvidovic, and Dashofy, 2009].

A formal definition of RA has not yet been determined so this paper suggests the following definition based on these below-mentioned attributes:

Definition 2. *An abstract domain-specific set of architectural decisions extracted from existing architectures of that domain that transform implicit knowledge to explicit knowledge and act as a configurable baseline to support system architecture construction and evolution.*

This definition connotes a very straightforward process for the creation and evaluation of RA:

Table I. Quality-Attribute Scenarios for Mobile SOAs

Quality Attribute	Stimulus	Response
Availability	Service consumer and the service provider leave access scope of each other.	Service continuity from the user's perspective.
	Service provider is brought to a halt.	
	Network connection of mobile device fails.	
	Connection intermittence occurs.	
Performance	User wishes to utilize a service.	Average response time for mobile service consumers for a specific service is less than double the response time for non-mobile service consumers of that service.
	The number of consumers accessing a mobile host is accrued.	The system is scalable to increase the user count.
Usability	User wishes to utilize a service.	The system renders a user interface, which uses the device specific facilities of the user's device such as NFC reader regarding the user's location.
Security	Access to transmitting data.	Information mapped to transmitting data is not accessible.
	Mobile device is lost or stolen [Natchetoi, Kaufman, and Shapiro, 2008].	Information mapped to stored data is not accessible.
Maintainability	Vendors introduce new mobile devices.	New devices can incorporate services affecting how they are used (service consumption) as well as provision and orchestration of services. Changes are very small.
Self-adaptation	Network topology and available services are changed.	A mobile device is aware of the available services or can discover them. In addition, service continuity is held from the user's perspective.

- Envisioning and determining architectural drivers.
- Pattern extraction.
- Developing a configurable baseline.
- Evaluating RA and its possible instances.

Proposed mobile SOA RA itself followed this process in the iterative manner.

3. ARCHITECTURAL DRIVERS

The following considerations influence the function of mobile devices as service consumers or service providers:

- Relocation of a service consumer or a service provider can change topology of communication network.
- Mobility can hinder access to previously available services and facilitate access to other environmental services. Hence, service discovery in a dynamic topology network of a mobile device is not as easy as it is for networks with fixed topologies [Ou et al., 2008; Reyes et al., 2008].
- Maintaining the availability of a service when a mobile device roams between heterogeneous wireless networks is a key consideration [Autili, Caporuscio, and Issarny, 2009].
- Mobile devices have low processing power that can be variable according to the mobile device being used.
- Mobile devices have limited memory size that can be variable according to the device being used.
- Mobile devices have limited bandwidth, variable according to the network being used.
- Mobile devices generally experience intermittent connectivity, some studies such as Philips, Straeten, and Jonckers [2013] consider (temporary) network failure as "the rule rather than the exception in nomadic (mobile) networks."
- In comparison with desktop computers, mobile devices have smaller keypads. Hence, user interfaces (UIs) designed for desktop computers can be inappropriate for mobile devices.
- In comparison with desktop computers, mobile devices have smaller screens. Hence, UIs designed for desktop computers can be inappropriate for mobile devices.
- Limited battery life of mobile devices poses considerations of power supply and energy consumption. While some mobile devices can provide auditory and sensing information in power saving mode without activating the main processor of the device [Chujo et al., 2013], service invocation involves processing and communicating that both consume battery power.
- Very few mobile operators provide public IPs for addressing mobile hosts [Srirama, Jarke, and Prin, 2006].
- Mobile devices usually have many network interfaces such as WiFi, Bluetooth, and GPRS compared to personal computers.
- Mobile devices have considerable potential for augmented reality UIs [Broll et al., 2007, 2009].

These aforementioned requirements for application in architectural drivers are shown in Table I.

4. MOBILE SOA PATTERNS

4.1. The Concept of Pattern

"A pattern can be thought of as a set of constraints on an architecture—on the element types and their patterns of interactions—and these constraints define a set or family of architectures that satisfy them" [Bass, Clements, and Kazman, 2003; Stricker et al., 2010]. Each pattern can handle one or more architectural drivers that may affect one or more quality-attributes [Bachmann, Bass, and Nord, 2007].

Each pattern describes the core of the solution to a recurring problem in an environment. Solutions are found in such a way that the user of the pattern can solve her/his problem by adapting the solution to her/his preferences that are also matched to local conditions [Haskins, 2008]. Patterns "are usually flexible and optional" and they may suggest alternative approaches [Erl, 2008]. "Instead of using patterns exactly as documented, they are meant to be a starting point from which the final product will be crafted. [...]. The benefit of this approach to patterns is that they should not remain stagnant, but instead be improved through continuous application" [Cloutier and Verma, 2007]. Patterns improve communication between various stakeholders [Cloutier and Verma, 2007], facilitate collaborative work [Pfister et al., 2012], and facilitate reuse which reduces the effort required for system testing, integration and maintenance [Cloutier and Verma, 2007]; they reportedly improve product quality and save time [Pfister et al., 2012]. Such patterns may also help control the complexity of architectures [Cloutier and Verma, 2007]. The remainder of this section briefly describes four concepts relating to patterns.

A compound pattern is a pattern comprised of a set of patterns. Even some core pattern within a compound pattern may be compound patterns in their own right [Erl, 2008]. Enterprise Service Bus (ESB) is an example of a (nested) compound SOA pattern [Erl, 2008]. Groups of related patterns (a pattern group) such as Service Definition Patterns may or may not constitute a compound pattern [Erl, 2008]. A pattern language is a set of complementary patterns [Cloutier and Verma, 2007]; it may allow patterns to combine in a variety of sequences (open-ended pattern language), or it may suggest the application sequences (structured pattern languages) [Erl, 2008]. A pattern catalogue is a documented set of related patterns [Erl, 2008]. The next section introduces a mobile SOA pattern catalogue, which provides an open-ended pattern language for mobile SOAs.

4.2. Mobile SOA Pattern Catalogue

This section presents identified mobile SOA patterns in an informal manner because a full description of them would significantly increase the length of this paper. Extracted patterns from existing mobile SOAs are as follows:

4.2.1. Pattern a: Prefetching

Prefetching is the term that describes a group of related patterns. Prefetching provides offline access to the service results that conceals connection intermittence and network latency from a user's perspective. Hence prefetching improves availability and performance of a system, prefetching, together with service advertisement, can eliminate service selection time from the user's point of view improving its efficiency and usability. This pattern group (prefetching) consists of the following patterns: peer-to-peer prefetching based on statistical data [Chang, Ling, and Krishnaswamy, 2011], consumer side prefetching [Liu and Deters, 2007] based on workflows defined by Business Process Execution Language (BPEL) files, and provider side prefetching based on statistical data [Natchetoi, Kaufman, and Shapiro, 2008; Liu and Deters, 2007; Schreiber et al., 2011], data mining [Tseng and Lin, 2006], and proximity relations [Eikerling, Benesch, and Berger, 2007]. In provider side prefetching, prefetched service results and computed hit ratio can be sent during a connection's idle time [Natchetoi, Kaufman, and Shapiro, 2008] or by piggybacking [Schreiber et al., 2011]; the latter makes a trade-off between energy consumption and response time [Schreiber et al., 2011].

4.2.2. Pattern b: UI Adaptation

Pattern b suggests customizing the UI of a mobile service for consumers based on statistical data, User Centered Design (UCD) guidelines and psychological models. The appropriateness of an UI of a specific mobile device depends on the facilities provided by the mobile device as well as the user's personality and the environment [Sefid-Dashti and Habibi, 2011]; as such UIs can be customized according to a combination of statistical data, UCD guidelines and psychological models [Sefid-Dashti and Habibi, 2011]. Some of the possible UIs are oral interfaces [Griol et al., 2009], augmented reality UIs [Broll et al., 2007; Broll et al., 2009] such as Near Field Communication (NFC) for touching and Visual Markers for pointing, and Conceptual Bookmarks [Henze et al., 2008]; in addition voice and text combinations such as Microsoft Live search [Acero et al., 2008] are among the more common options. This pattern improves usability and makes a compromise between response time and fully fledged UI.

4.2.3. Pattern c: Transformations and Simplifications

"Transformations and simplifications" is the term that describes a group of related patterns. The main purpose of this pattern group is to optimize performance. Enhanced performance can be achieved by reducing the consumption of resources by transforming and simplifying invocation data. These transformations and simplifications are applied to data used by the service, relating to data storage and data transfer between service parties. Transformations such as streaming serve to reduce data transmission and consumption of resources in order to eliminate processes of serialization and deserialization of SOAP messages [Oh and Fox, 2007]. Examples of transformations and simplifications used by this pattern group are as follows; lossy-compression (based on knowledge of a business process) [Natchetoi, Kaufman, and Shapiro, 2008]; lossless-compression [Papageorgiou et al., 2010]; retrieving an equivalent form of each Simple Object

Access Protocol (SOAP) message [Apte, Deutsch, and Jain, 2005]; structural metadata transmission and compression based on transmitted metadata [Wu and Natchetoi, 2007]; converting service invocation results to JavaScript Object Notation (JSON) that requires less data in terms of size and client side processing than XML [Hamdi, Wu, and Dagtas, 2008]; using compressed versions of agent protocols such as ACL and FIPA-SL instead of SOAP and Ontology Web Language (OWL) [Shajjar et al., 2008].

Tian et al. [2004] considered net times of service invocation excluding compression/decompression times and waiting-times.

4.2.4. Pattern d: Preprocessing and Filtering

Preprocessing of service requests (filling default parameters by proxy [Schreiber et al., 2011]) and filtering service invocation results [Schreiber et al., 2011; Hamdi, Wu, and Dagtas, 2008] are processes dealt with by *pattern d*.

4.2.5. Pattern e: Service Interactions Aggregation

Pattern e suggests that users send all interaction data to a system in a single request; the agents then do parallel interactions within the composite service on behalf of the mobile user and then send results of that service invocation back to the user [Zahreddine and Mahmoud, 2005]. This pattern serves to reduce transmissions of metadata and unifies duplicated input values of a sequence of messages.

4.2.6. Pattern f: SOAP Processing Cost Migration

Pattern f recommends eliminating the SOAP processing cost from mobile consumers by doing interactions on behalf of the user via software agents [Adaçal and Bener, 2006] or a gateway [Reyes et al., 2008].

4.2.7. Pattern g: Protocol Scale-Down

Protocol scale-down describes a group of related patterns. *Pattern g* supports mobile devices with several constraints on their resources by recommending a subset of specifications for common protocols such as Limited SOAP (engine) for mobile service providers [Pawar et al., 2007], limited SOAP for mobile service consumers [Natchetoi, Kaufman, and Shapiro, 2008], limited BPEL for mobile service consumers [Natchetoi, Kaufman, and Shapiro, 2008] and authentication of a mobile user based on the lower half of the iris (eye) [Al-Hussain and Al-Rassan, 2010].

4.2.8. Pattern h: Parsimonious Asymmetric Encryption

Pattern h recommends using asymmetric encryption for the more sensitive information and using symmetric encryption for all other information [Natchetoi, Kaufman, and Shapiro, 2008; Srirama, Jarke, and Prin, 2006]. It makes a trade-off between security and performance of mobile SOAs because the processing cost of asymmetric encryption is beyond the resources available in the majority of mobile devices.

4.2.9. Pattern i: Asynchronous Service Invocation

Asynchronous service invocation loosely matches a coupled sender and receiver within a mobile environment. *Pattern i* recommends asynchronous service invocation for allowing applications to function in a disconnected mode [Natchetoi, Kaufman, and Shapiro, 2008; Elgazzar, Martin, and Hassanein, 2012]. Asynchronous service invocation is more scalable than synchronous service invocation and does not block the UI thread it thereby improves usability of a system [Singh, Mishra, and Kushwaha, 2009]. Asynchronous service invocation in mobile SOAs is realized by a queue [Hamdi, Wu, and Dagtas, 2008; Wu and Natchetoi, 2007] or a gateway [Singh, Mishra, and Kushwaha, 2009] so that tasks consuming resources such as callback or polling are delegated.

4.2.10. Pattern j: Connection Management and Merging

The various network channels of mobile devices have different capabilities [Natchetoi, Kaufman, and Shapiro, 2008; Papageorgiou et al., 2012a]. Selections are made with a view to improving performance, service availability and energy efficiency. This can be achieved by selecting a communication channel for service discovery for a specific service consumer. Furthermore, substituting connections and merging all possible channels such as WiFi, Bluetooth, and GPRS, are dealt with by this pattern (pattern group) according to a process facilitated by the following:

- Message break up: the sender breaks up the message into short packets to accommodate any limitations of channels; the receiver then merges those packets [Wu and Natchetoi, 2007].
- Energy-efficient network configuration [Caporuscio, Charlet, and Issarny, 2007].
- Using convergence of cellular and *ad hoc* networks [Ou et al., 2008].
- Connecting to the next available network before disconnecting from a currently connected network [Autili, Caporuscio, and Issarny, 2009].
- Connection substitution: substituting communication channels in a single session based on changes to available networks: for example switching transparently between Bluetooth and GPRS in close proximity to a computer [Natchetoi, Kaufman, and Shapiro, 2008].

Caporuscio, Raverdy, and Issarny [2012] introduced the use of a unique ID for each application. And each ID uses several network interfaces by offering mapping of that ID to the actual set of IP addresses assigned to the network interfaces. This mapping allows for a mobile host detached from a specific network to be available to consumers through other networks to which that mobile host is connected.

Papageorgiou et al. [2012a] analyzed levels of energy consumption of mobile clients during web service invocation; the research suggests that attempts be made to reduce time intervals between (multiple) invocations and to increase

temporal overlapping of the tasks of transmission and processing. The research also considers UMTS's device-network tail times and 3G energy consumption in periods in which no data transmission occurs.

4.2.11. Pattern k: Multinetwork Routing

Pattern k suggests using multinetwork routing [Autili, Caporuscio, and Issarny, 2008] to enable access to services that may be in separate networks to which the hosts have network-interfaces, which may not be compatible with those of the consumer's mobile device.

4.2.12. Pattern l: Network Infrastructural Path

Autili, Caporuscio, and Issarny [2008] reported that an uninterrupted network path is achieved in circumstances when there is at least one network path in common between the consumer and the service provider. An example of such a network infrastructural path would be WiFi1 $\Leftrightarrow$ UMTS1 $\Leftrightarrow$ WiFi2 [Autili, Caporuscio, and Issarny, 2008].

4.2.13. Pattern m: SOAP Transfer Protocol

Transferring SOAP messages over SMS instead of HTTP in a cellular network facilitates addressing [Singh, Mishra, and Kushwaha, 2009]. Kim and Lee [2009] recommend transferring those messages over WAP and Bluetooth for channel management [Kim and Lee, 2009].

Transferring SOAP messages over these mentioned protocols rather than over HTTP is the protocol considered in this pattern (pattern group).

4.2.14. Pattern n: Mobile Service Composition

Mobile service composition is a group of related patterns. Service composition extensions for mobile SOAs include personalized service composition [Jørstad, Thanh, and Dustdar, 2004], dynamic service composition [Philips, Straeten, and Jonckers, 2013; Thanh and Jørstad, 2005], *ad hoc* service composition [Ni and Sloman, 2005], goal-driven service composition [Ni and Sloman, 2005], and adaptable service composition based on mapping the service model of Semantic Markup for Web Services (OWL-S) to actions of agents [Cao and Li, 2009] and finally, service composition based on historical colocation patterns of a service consumer and service providers [Prete and Capra, 2008].

Each mobile service composition extension intends to enhance the availability of a composite service except for personalized service composition, which intends to enhance usability.

4.2.15. Pattern o: Mediator

Pattern o suggests a mediation framework or a mediation element to cope with the resource constraints of mobile devices. The mediator responsibilities involve service invocation on behalf of a mobile service consumer [Reyes et al., 2008], WS-Security processing of a mobile service provider [Srirama, Jarke, and Prin, 2006, 2007; Asif, Majumdar, and Dragnea, 2008], WS-Policy processing of a mobile service provider [Asif, Majumdar, and Dragnea, 2008], using agents to serve as a mobile proxy to continue service interactions regardless of mobile device disconnections [Leong et al., 2007], and to distribute service invocation tasks with small granularity such as discovery, extraction, selection, installation, invocation, and presentation of a service's output [Duda, Aleksy, and Butter, 2005].

4.2.16. Pattern p: Distributed Peer-to-Peer Registry

Available services in a mobile environment change more frequently than those of enterprise SOAs, hence centralized registries are not considered appropriate for mobile SOAs. In contrast to centralized registries, distributed registries are more scalable; distributed registries are based on protocols of peer-to-peer networks common in mobile SOAs. A pastry overlay network has a discovery time in the order of nLogn for n nodes of the overlay network [Reyes et al., 2008], a layered overlay network [Ou et al., 2008], and JuXTApose (JXTA) [Srirama, Jarke, and Prin, 2006] used in the distributed peer-to-peer registries of mobile SOAs.

4.2.17. Pattern q: Peer ID Instead of Public IP

The problem of addressing mobile hosts for which their mobile operators do not provide public IPs can be overcome by using the unique peer ID of a peer-to-peer network [Srirama, Jarke, and Prin, 2006].

4.2.18. Pattern r: Service Migration and Replication

Available resources of mobile hosts such as energy (battery), memory, and bandwidth can change over time. Services can migrate to or replicate hosts with sufficient resources to maintain service availability and performance (response time and scalability).

Sheng et al. [2009] categorize services in terms of their being mobile or stationary; the report maintains that stationary services depend on the resources of their host such as those of a database, so these services cannot migrate to other hosts. Tanenbaumn and Steen [2006] categorize distributed-system resource's conditions of migration based on process-to-resource binding and resource-to-machine binding in nine categories. But the constrained resources of mobile SOAs narrow the options for migration as categorized by Sheng et al. [2009].

Service replication, migration [Kim and Lee, 2009; Marzouk et al., 2009; Schmidt et al., 2007; Sheng et al., 2009], and migration-policy [Kim and Lee, 2009] improve the availability and performance of serving but they need to consider addressing requirements [Schmidt et al., 2007; Sheng et al., 2009].

4.2.19. Pattern s: Deploying Application Equivalent to a Service

Replicating the most frequently used mobile services used by a client can remedy any connection problems experienced by a mobile user but this needs to be a more computational resource than the usual service invocation and is not size scalable; running an equivalent application needs less resources than running the service [Lee et al., 2011] as a host of that service. *Pattern s* suggests incorporating applications that function in a similar way to the service on the mobile device; Lee et al. [2011] introduce a solution for this pattern using Android class loading.

4.2.20. Pattern t: Context Awareness

Context aware SOAs [Truong and Dustdar, 2009] are a subset of SOAs that intersects with mobile SOAs. Service adaptation by predefined adaptive behavior for some possible changes in context [Bellur and Bondre, 2008] is an example of a context aware solution.

In mobile SOAs, there are three types of contexts; those of consumers, providers and networks [Autili, Caporuscio, and Issarny, 2008; Chang, Tseng, and Lin, 2008] that need to be considered when dealing with mobility problems.

4.2.21. Pattern u: Semantic Services and Dynamic Service Adaptation

Defining domain specific ontology and using the standard ontology such as OWL-S and FIPA-SL for dynamic service composition and dynamic service adaptation is becoming increasingly common [Cao and Li, 2009; Ni and Sloman, 2005].

4.2.22. Pattern v: Search Engine Instead of Registry

Pattern v suggests the scalable discovery of semantic services by a search engine instead of the more commonly used registries [Liu and Connelly, 2008].

4.2.23. Pattern w: Integrating the UI of a Service Into the UI of A Mobile Device

Pattern w suggests integrating the UI of a service into the UI of a mobile device: for example integrating the services into menus to offer a faster route to them. This pattern is device specific [Gurp, Karhinen, and Bosch, 2006]; hence it is not a scalable solution while solutions such as *pattern b*, that use a parameterized approach are more scalable than this pattern.

4.2.24. Pattern x: On Demand Mobile Host Startup

"On demand" management of mobile host resources is crucial to provide scalability to mobile hosts. This pattern uses a mediator to facilitate mobile host start up on the first request [Srirama, Jarke, and Prin, 2006, 2007].

4.2.25. Pattern y: Distributed Cache of Service Contracts

Caching service results can be utilized when future requests use arguments identical to the cached response. Paul et al. [2013] suggest caching service contracts to avoid repetitive retrieve of the same service contract from registry. They use a Distributed Spanning Tree (DST) overlay structure, which is built on the top of mobile peer-to-peer network so as to reduce the number of hops required to reach the node which contains the cached WSDL file.

4.2.26. Pattern z: Inquiring about Cache Freshness

Besides caching of service contracts (*pattern y*), client (consumer) side caching of web service results can, in parts, spare retransmission of service invocation messages. In addition, client side caching is a vital ingredient in prefetching (pattern a). *Pattern z* considers inquiring freshness of cached service results before use of the cached service results.

Papageorgiou et al. [2012b] define a cache mediator, which generates cache aware service proxy from service description (Unified Service Description Language) so that mobile consumers can use these service proxies which are deployed in containers. Proxied consumption of service includes invocating the service in IT systems over fast connections and returning either confirmation of freshness of the cached service result or updated service results of the invocated service. Depending on size of the service invocation results receiving the confirmation can reduce the size of exchanged messages.

Ensuring freshness of cached service results through (cache aware) proxied consumption of services is considered in this pattern.

In addition to these mentioned patterns, RESTful Web Services [Rodriguez, 2008; Vinoski, 2007] are discussed in some mobile service-based systems [Chang, Mohd-Yasin, and Mustapha, 2009; Liu and Connelly, 2008; Li and Sun, 2009]. In comparison with SOAP, REST has shorter messages and less complex architecture [Chang, Mohd-Yasin, and Mustapha, 2009]. The proposed RA may be adapted to REST, although REST and SOA are two distinct architectural styles and many of these aforementioned patterns are SOAP dependent.

4.3. Architectural Trends in Mobile SOAs

The catalogue of mobile SOA patterns implies that mobile SOAs have fixed trends, aka themes, in their architectural decisions. These trends and patterns are shown in Table II. For example the trend *selective communication models* is associated with patterns *a*, *i*, *y*, and *z pattern* that constitute an asynchronous persistent communication. An architect may use a proposed pattern(s) or another solution in each trend. Note that *pattern v* and *pattern w* are not associated with any specific trend.

A considerable number of mobile SOA patterns use location information. Hoober and Berkman [2011] discuss methods of retrieving location information (Cell, Sector, Triangulation, GPS telemetry, WAAS, AGPS, WLANs, and PANs) that can be used to implement those patterns.

Table II. Architectural Trends in Mobile SOAs

Trend	Patterns
Look ahead service provision	a, n
Intelligent adaption and composition	b, n, t, u
Parsimonious resource consumption and on demand resource consumption	c, d, e, h, j, p, x
Mediation, preprocessing and task delegation	d, e, f, i, k, o, r, x, z
Service virtualization	a, r, s
Context awareness	b, n, t
Semantic services	n, u
Service interaction task delegation	d, f, i, o
User interaction task delegation and cost migration	e
Subsetting	g
Selective communication models	a, i, y, z
Peer to Peer addressing and discovery	p, q, y
Connection management and multiradio networking	j, k, l, m, t
Service migration and replication	r, s, y

5. MOBILE SOA RA

Mobile SOAs facilitate mobile device functions that operate as service participants; hence mobile SOA RA is considered as an extension of SOA RA [Arsanjani et al., 2007; Open Group, 2009, 2011; Zhang and Zhang, 2010]. A well-accepted family of SOA RA [Arsanjani et al., 2007; Open Group, 2009, 2011; Zhang and Zhang, 2010] was introduced by IBM SOA Solution Stack (S3) [Arsanjani et al., 2007] and then finished by the Open Group SOA RA [Open Group, 2011]. The mobile SOA RA proposed in this paper is an extension of that study, in that it includes all nine layers of this family of RA. This family will hereafter be cited within the text as SOA RAF. This mobile SOA RA adds a new domain specific layer to SOA RAF and distributes mobile SOA patterns among the layers (10 layers = 9 layers inherited from SOA RAF and a new domain specific layer). Figure 1 depicts this proposed RA in 10 logical layers, presenting a partially layered architecture [Open Group, 2009]; hence each layer can have more dependencies than its immediate successor in a layered style: for example a consumer has threefold access options—services of the *Business Process Layer*, services of the *Services Layer*, and prefetched services of the *Prediction Methods and Statistical Data Layer*, but it does not have access beyond the constraints of the SOA architectural style. It is worth highlighting that SOA RAF uses the term "layer," which also is the name of a domain-independent architectural pattern, for both layers and crosscutting concerns and this paper follows this use of the term "layer" to facilitate understanding the maps between SOA RAF and mobile SOA RA. Note that Figure 1 uses the informal notation of SOA RAF [Arsanjani et al., 2007; Open Group, 2009, 2011; Zhang and Zhang, 2010] instead of formal notations such as Service oriented architecture Modeling Language (SoaML); merely boxes represent layers, ellipses represent services, solid lines represent UML/SysML association, dashed lines represent UML/SysML dependency, script symbols represent rules and other symbols intuitively represent components. This notation is common in SOA RAF with some exceptions that are considered irrelevant.

The proposed RA has a layer which does not have a corresponding SOA layer (the *Prediction Methods and Statistical*

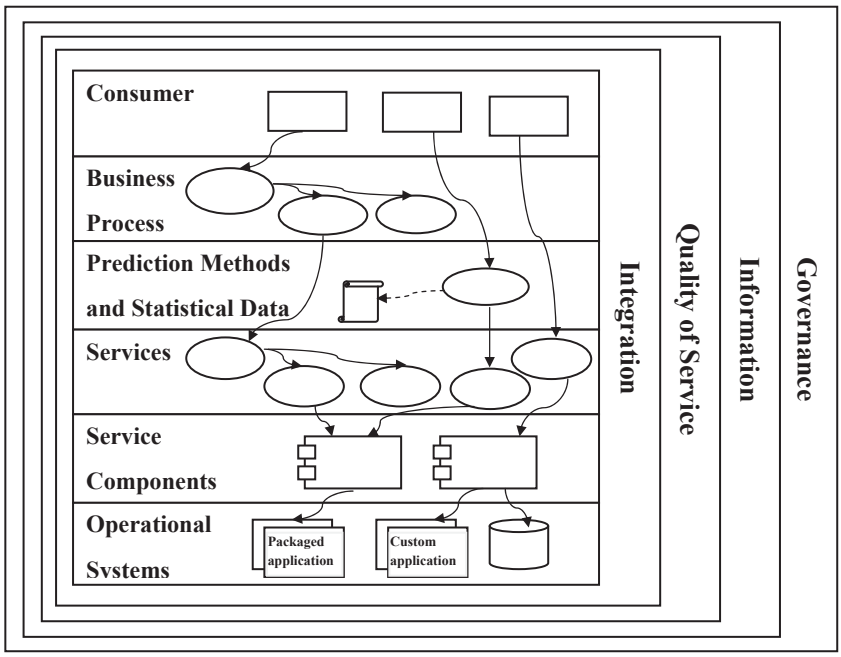

Figure 1. Logical solution view of the mobile SOA RA.

Data Layer); it has six extended-layers (the *Operational Systems Layer*, the *Services Layer*, the *Business Process Layer*, the *Consumer Layer*, the *Integration Layer*, and the *Quality of Service Layer*) and three of its layers match the corresponding layers in SOA RAF (the *Service Components*, *Information* and *Governance* layers). Table III shows the distribution of mobile SOA patterns among the layers; it shows that the concerns that each pattern addresses are aligned with definitions, abilities and Architectural Building Blocks (ABBs) of a layer. It is noteworthy that patterns associated with the *Governance Layer* and the *Information Layer* facilitate realization of (some) abilities of these layers regarding mobile SOAs, but these patterns do not extend the responsibilities of these layers.

Patterns offer solutions for realizing abilities mentioned in the Open Group SOA RA: e.g., the *pattern r* (Service Migration and Replication) uses checkpoints [Marzouk et al., 2009] to realize abilities such as the ability to account for successful operations [Open Group, 2011] mentioned in the *Quality of Service* layer of the Open Group SOA RA. The *pattern r* is also a realization of the ability to make adaptations to a configuration so as to ensure the quality of a service [Open Group, 2011] mentioned in the *Quality of Service* layer of the Open Group SOA RA.

Similar to SOA RA [Arsanjani et al., 2007; Open Group, 2009, 2011], three layers of mobile SOA RA address its implementation and interface with services (the *Operational Systems Layer*, the *Service Components Layer*, and the *Services Layer*). Three of them support the consumption of services (the *Business Process Layer*, the *Prediction Methods and Statistical Data Layer*, and the *Consumer Layer*). Four of these layers: *Information*, *Quality of Service*, *Integration*, and *Governance* layers concentrate on crosscutting-concerns.

Each layer has been described only briefly along with tasks necessary to facilitate the construction and evolution of concrete mobile SOA solutions. More detailed description of each SOA layer that include their specific responsibilities is beyond the scope of this work but are reported elsewhere in SOA RAF [Arsanjani et al., 2007; Open Group, 2009, 2011; Zhang and Zhang, 2010]; hence descriptions in this paper are concentrated on mobility in SOA.

Table III. Distribution of the Mobile SOA Patterns Among the Layers

Layer	Mobile SOA Patterns
Governance	p, q, r, v
Information	z
Quality of Service	h, l, r, t, x
Integration	c, d, e, f, g, h, i, j, k, l, m, o, p, q, r, s, u, v
Consumer	a, b, w, z
Business Process	n
Prediction Methods and Statistical Data	a, b, n
Services	a, r, u, y
Service Components	
Operational Systems	j, k, l, r, s

5.1. Operational Systems Layer

This layer includes all physical, infrastructural and application assets needed to support the SOA solution [Open Group, 2009]. In mobile SOA solutions this layer of responsibilities is extended to manage multichannel connection [Autili, Caporuscio, and Issarny, 2008, 2009; Caporuscio, Charlet, and Issarny, 2007; Caporuscio, Raverdy, and Issarny, 2012; Natchetoi, Kaufman, and Shapiro, 2008; Ou et al., 2008; Wu and Natchetoi, 2007]; network infrastructural paths [Autili, Caporuscio, and Issarny, 2008] and physical support for service migration [Kim and Lee, 2009; Lee et al., 2011; Marzouk et al., 2009; Schmidt et al., 2007; Sheng et al., 2009].

5.2. Service Components Layer

The service components that realize services and their functioning reside in this layer [Open Group, 2009]. The layer serves to regulate the service by aligning the components that facilitate IT implementation with the service description. This is realized by each service presenting a component that is open to change by the IT operation in the layer [Open Group, 2011].

This layer is identical to the *Service Components Layer* of the SOA RAF and it has no associated mobile SOA pattern.

5.3. Services Layer

This layer represents services defined in the SOA [Open Group, 2009]. Semantic services are increasingly commonplace in SOAs. Semantic services [Cao and Li, 2009; Ni and Sloman, 2005; Shajjar et al., 2008] and ontology are key factors for various extensions to services (adaptive [Cao and Li, 2009], dynamic [Philips, Straeten, and Jonckers, 2013; Prete and Capra, 2008; Thanh and Jørstad, 2005], *ad hoc* [Ni and Sloman, 2005], and goal-driven [Ni and Sloman, 2005]) in mobile SOAs. The *pattern y* (Distributed Cache of Service Contracts) supports scalability of cached services contracts; in addition, the *pattern a* (prefetching) and the *pattern r* (Service Migration and Replication) both involve caching and clustering so they are partly responsible for realization of *Cluster Manager* ABB of the Open Group SOA RA.

5.4. Prediction Methods and Statistical Data Layer

As mentioned, this layer has no corresponding layer in the SOA RAF; some responsibilities of this layer may be realized as a middleware [Schreiber et al., 2011]. As patterns associated to this layer (see Table III) imply two main responsibilities the first is to optimize resource consumption based on predictable sequences and the second is to adapt service interactions to enhance a user's experience and to provide

service continuity. So this layer serves to improve availability, performance and usability.

The majority of mobile SOA patterns in other layers (the nine layers in Open Group SOA RA) have corresponding ABBs in the Open Group SOA RA: e.g., the *pattern r* (Service Migration and Replication) is a solution for realizing Cluster Manager ABB in the Services Layer and is a solution for realizing part of responsibilities of Availability Manager ABB in the Quality of Service Layer. But those patterns that reside in *Prediction Methods and Statistical Data Layer* are not realizations of any ABB of Open Group SOA RA. Patterns associated with this layer add intelligent behaviour to mobile SOAs in both personalization (UI adaptation and personalized service composition) and look ahead at service provisioning (prefetching and mobile service composition). The ABB of the Open Group SOA RA are briefly cited as ABB throughout the paper.

This layer provides information on personalization for *Presentation Adapter* ABB in the *Consumer Layer*; it handles a client's personal preferences by processing the relevant decompositions and compositions [Open Group, 2011].

The *Prediction Methods and Statistical Data Layer* uses four types of data for inputs: (1) the statistical data related to historical service invocations, locations of invocations and their times. (2) Data mining on these statistical data [Tseng and Lin, 2006]. (3) Results of personality analysis on conversation logs and SMS logs of a user [Sefid-Dashti and Habibi, 2011]. (4) Data achieved from public services such as regulations [Sefid-Dashti and Habibi, 2011]. This layer uses data from various sources to make predictions for service requests and to prefetch it into a consumer's mobile device [Chang, Ling, and Krishnaswamy, 2011; Eikerling, Benesch, and Berger, 2007; Liu and Deters, 2007; Natchetoi, Kaufman, and Shapiro, 2008; Schreiber et al., 2011; Tseng and Lin, 2006]. It also uses data to provide or generate UI [Broll et al., 2009; Sefid-Dashti and Habibi, 2011] appropriate for the facilities of the mobile device, a user's personality and the environment [Sefid-Dashti and Habibi, 2011]. Finally, this layer handles mobile SOA extensions of service composition (personalized [Jørstad, Thanh, and Dustdar, 2004], adaptive [Cao and Li, 2009], dynamic [Philips, Straeten, and Jonckers, 2013; Prete and Capra, 2008; Thanh and Jørstad, 2005], *ad hoc* [Ni and Sloman, 2005], and goal-driven [Ni and Sloman, 2005]) that all serve to improve service continuity and usability.

5.5. Business Process Layer

Composite services that are analogues for significant business processes are defined in this layer [Arsanjani et al., 2007]. Consumer side prefetching [Liu and Deters, 2007], based on workflows is defined by BPEL files in this layer. Also, light weight workflow engine [Plebani et al., 2012] and supporting a subset of BPEL specifications [Natchetoi, Kaufman, and Shapiro, 2008] that allow more mobile devices are incorporated into service composition processed in this layer. As an alternative to BPEL, Philips, Straeten, and Jonckers [2013] propose a workflow language for mobile (nomadic) networks that provides default compensation actions (restarting the activity, rediscovering a service of the same type) and enables overriding and/ or extending the default compensation actions.

5.6. Consumer Layer

The *Consumer Layer* handles interactions with the user or with other programs in an SOA ecosystem according to specific user preferences [Arsanjani et al., 2007]. This layer has the capability to "quickly create the front end of business processes and composite applications" [Open Group, 2011]. Furthermore, the consumer layer is coordinated with the *Prediction Methods and Statistical Data Layer*, to provide logs of cache access and logs of chosen options of the UI to improve accuracy of prefetching and accuracy of generating UI, respectively. These feedback data can be sent by piggybacking [Schreiber et al., 2011] or in times of an idle connection [Natchetoi, Kaufman, and Shapiro, 2008].

5.7. Integration Layer

The *Integration Layer* is a key enabler for SOA. The layer functions as a routing mechanism, it can operate in terms of a simple point-to-point integrator to secure coupled endpoints or deal with more complex routing such as protocol mediation, and other transport capabilities sometimes described as ESB [Open Group, 2009].

Table III depicts that the majority of mobile SOA patterns reside in this layer, indicating a convergence of mobile SOA considerations. The main capability of the *Integration Layer* in mobile SOAs are as follows: transformations, supporting lightweight, and simplified protocols, preprocessing, interacting on behalf of mobile service consumers, interacting on behalf of mobile service providers, supporting multinetwork routing, supporting distributed registry, addressing mobile hosts with no IP address and supporting service migration and virtualization. These patterns correspond to ABBs such as *Mediator*, *Router*, *Protocol Converter*, and *Message Transformer* in Open Group SOA RA. The ESBs support most of those tasks; Optimizing ESBs for the eighteen associated mobile SOA patterns can form a mobile ESB concept that provides pattern implementation.

5.8. Quality of Service Layer

This layer enforces policies defined in the *Governance Layer*. It monitors and captures service and solution metrics in an operational sense and signals "noncompliance with NFRs relating to the salient service qualities and policies associated with each SOA layer" [Open Group, 2009]. This layer depends on context awareness to ensure quality [Autili, Caporuscio, and Issarny, 2008; Chang, Tseng, and Lin, 2008; Ni and Sloman, 2005]. It requires gathering information relating to the consumer, the provider and the network [Autili, Caporuscio, and Issarny, 2008; Chang, Tseng, and Lin, 2008]. The *pattern l* (Network Infrastructural Path) partly enables responses to changes in infrastructure to maximize availability and this ability is defined in the *Quality of Service Layer* of Open Group SOA RA [Open Group, 2011]. This layer optimizes

performance by its ability to support virtualization of resources [Open Group, 2011] of the *Performance Manager* ABB that may partly be realized by the *pattern r* (Service Migration and Replication).

5.9. Information Layer

The *Information Layer* unifies representations of information from different sources [Open Group, 2011]. The *pattern z* (Inquiring about Cache Freshness) ensures freshness of cached service results so it is partly responsible for realization of *Data Cache* ABB that supports consistency of updates and the availability of data. This layer has bidirectional interaction with the *Prediction Methods and Statistical Data Layer*. The *Traceability Enabler/Auditor* ABB in the *Information Layer* provides a traceability log which includes information on who has accessed the data, when, and details of data that has been accessed [Open Group, 2011]. The *Prediction Methods and Statistical Data Layer* uses a traceability log of the *Traceability Enabler/Auditor* ABB and uses trends and patterns identified by *Data Miner* ABB of the *Information Layer* for prefetching [Eikerling, Benesch, and Berger, 2007; Liu and Deters, 2007; Natchetoi, Kaufman, and Shapiro, 2008; Schreiber et al., 2011; Tseng and Lin, 2006], service composition [Prete and Capra, 2008], and analyzing a user's personality for UI adaption [Sefid-Dashti and Habibi, 2011]. In reverse direction the *Prediction Methods and Statistical Data Layer* sends cache access rates and success rates of UI navigation predictions to the *Information Layer* to accommodate its optimization algorithms and its analytical information.

5.10. Governance Layer

The layer responsible for SOA Governance ensures that the services and SOA solutions within an organization comply with the policies, guidelines and standards that are defined as a function of the objectives, strategies and regulations applied in the organization and that SOA solutions provide the desired business value [Open Group, 2011]. This layer includes all policies such as the migration policy for the *pattern r* and it is the logical location to serve as a repository and registry [Open Group, 2011] or distributed registries (the *pattern p* and the *pattern v*) of mobile SOAs.

An architect can instantiate a concrete architecture by weaving patterns of each layer based on the specific architectural drivers of her/his project. Other points of variations in proposed RA are its layers. For example, a given (mobile) SOA solution may exclude a *Business Process Layer* and have the *Consumer Layer* interacting directly with the *Service Layer*. "Such a solution would not benefit from the business value proposition associated with the *Business Process Layer* however that value could be achieved at a later stage by adding a layer" [Open Group, 2009].

6. EVALUATION

Knowing the architecture of a system will expose the important properties of that system [Bass, Clements, and Kazman, 2003]. The application of architectural design principles incorporates predetermined properties for the development of systems, so that design choices can be analyzed. The system can be understood and appreciated during its development and then tested before being realized [Bass, Clements, and Kazman, 2003]. Architectures of diverse systems of a domain may be instantiated from a RA. Hence evaluation of RA before its adoption by the stakeholders is even more important than evaluation of the architecture of a system of that domain. Evaluation of system architecture includes low-cost risk mitigation, and well-evaluated RA fosters rational incentives for wider adoption of that particular RA.

Patterns evolve and are then discovered as opposed to being invented [Ackerman and Gonzalez, 2010; Cloutier and Verma, 2007]; hence they are proven elements of the RA but the RA itself and its possible instantiation must be evaluated. For each RA there are two sets of stakeholders: the first set (architects) instantiates architecture from the RA for development of a specific system, and the second constitutes stakeholders of the system. RA states commonalities and encompasses points of variations [Nakagawa, 2012]. Variability is the basis of different forms of instantiable (software) architecture but generating a complete collection of possible architectures is impossible for continuous values of quality attributes and it also is infeasible for discrete values of quality attributes because it leads to combinational possibilities. In conclusion, the evaluation of RA poses two challenges [Angelov, Trienekens, and Grefen, 2008]:

- Lack of access to a complete set of stakeholders and more accurately, lack of access to the stakeholders of futuristic systems of a domain.
- RA is inherently abstract hence there may be a huge number of scenarios for evaluating instantiable architectures.

Regarding these challenges, the mobile SOA RA and its possible instantiation was evaluated by Architecture Tradeoff Analysis Method (ATAM) [Angelov, Trienekens, and Grefen, 2008; Bass, Clements, and Kazman, 2003; Gallagher, 2000] (Section 6.1.) and the method proposed by Kazman et al. [2011] (Section 6.2.), respectively.

6.1. Evaluation of Mobile SOA RA by ATAM

Methods to evaluate software architecture cannot be applied directly to the evaluation of RA [Angelov, Trienekens, and Grefen, 2008]. The differences between RA and system architecture were discussed in Section 2. There are some adaptations of ATAM for evaluations of RA [Angelov, Trienekens, and Grefen, 2008; Gallagher, 2000].

In evaluation of mobile SOA RA by ATAM, as suggested by Angelov, Trienekens, and Grefen [2008], a sample set of stakeholders was involved; they were representatives of the stakeholders of instantiable architectures. According to Gallagher [2000] stakeholders of the RA itself should be used; hence three people with knowledge of software architecture were added to the set of stakeholders. Regarding the second challenge, this evaluation used both general and concrete scenarios. Table IV shows parts of the utility tree including general scenarios and corresponding examples of concrete

Table IV. Parts of the Utility Tree

Quality Attribute	Attribute Refinement	General Scenario	Concrete Instance of the General Scenario	Importance/ Realization Difficulty	Scenario No.
Performance	Response time	In a normal operation, average response time for mobile service consumers for a specific service is less than double the average response time for nonmobile service consumers of that service.	Response time for invoking service X (e.g.: personal images) of a mobile service provider by a mobile service consumer is less than two seconds.	High, high	1
		A service consumer experiences the same response time in several invocations of the same service of the same provider in a session.	The system covers 99.99 of the jitters.	Medium, high	2
	Scalability	The number of users of the system increases and the system is scalable to increase the user count.	Concurrent users of a mobile service provider, which provides service X increases from 5 to 14; so that service X is provided without change in response time for the consumer.	High, high	3
		In spite of the huge number of possible services and continuous changes in availability of those services, service discovery is scalable.	Service discovery for 700 services from 200 mobile service providers and 100 nonmobile service providers takes less than 7 seconds for a mobile service consumer.	High, medium	4
Maintainability	Extensibility	When several mobile operators support ultra-broadband internet access in fourth generation mobile networks, the system can integrate with cloud computing solutions to cope with any constraints on memory and processing power of mobile devices. This integration requires little effort.		Medium, high	5
Availability		The service provider of a simple service is brought to a halt; the system has failure-transparency and the user is conducted to consume the service from another provider so that the user remains unaware of this change.	Failure of the service provider of service X is concealed from the user and service continuity is achieved in 5 seconds at the most.	High, medium	6
Usability		An employee experienced with a nonmobile version of a system can become a professional user of a mobile version of that system.	An employee who has worked a year with system Y, which encompasses services A, B and C learns the mobile version of the system and works as a professional user of the mobile version after three days.	Low, low	7
Security	Integrity	The system persists against unauthorized intrusions.		High, medium	8

Scenario no.: 1	General scenario: average response time for mobile service consumers for a specific service is less than double the average response time for non-mobile service consumers of that service. Concrete instance: response time of invoking service X (e.g.: personal images) of a mobile service provider by a mobile service consumer is less than two seconds.			
Attribute	Performance - response time			
Environment	Normal operations			
Stimulus	A mobile service consumer invokes a service hosted on a mobile or non-mobile host.			
Response	The request is responded with short response time.			
Architectural decisions	sensitivity	Trade-off	Risk	Non-risk
Pattern a - Pre-fetching	S1	T1		
Pattern c - Transformations and simplifications	S2			
Pattern d - Pre-processing and filtering				
Pattern e - Service interactions aggregation				
Pattern j - Connection management and merging		T2		
Pattern o – Mediator - distributing service invocation tasks				
Pattern z – Inquiring about Cache Freshness				
Reasoning	• Attempting to use all available bandwidth through the *pattern a*, the *pattern c* and the *pattern j*. • To reduce required bandwidth for service invocations through the *pattern d*, the *pattern e* and the *pattern o*. • Ensuring the freshness of a pre-fetched service when a cache hit occurs for pre-fetched service (whenever a cache hit for a pre-fetched service occurs, *pattern z* is used to ensure freshness of the pre-fetched service).			

Figure 2. Analysis of mobile SOA RA for scenario no. 5 of the utility tree.

Scenario no.: 5	General scenario: when several mobile operators support ultra-broadband internet access in fourth generation mobile networks, the system can integrate with cloud computing solutions to cope with any constraints on memory and processing power of mobile devices. This integration requires little effort.			
Attribute	Maintainability - extensibility			
Environment	Maintenance time			
Stimulus	Several mobile operators support ultra-broadband internet access of forth generation mobile networks and stakeholders request to integrate the system with a cloud computing solution.			
Response	Integration is performed with little effort.			
Architectural decisions	sensitivity	Trade-off	Risk	Non-risk
layering		T3	R1	
Reasoning	Prevention of ripple effects.			

Figure 3. Analysis of mobile SOA RA for scenario no. 1 of the utility tree.

Scenario no.: 6	General scenario: the service provider of a simple service is brought to a halt; the system has failure-transparency and the user is conducted to consume the service from another provider so that the user remains unaware of this change. Concrete instance: failure of the service provider of service X is concealed from the user and service continuity is achieved in 5 seconds at the most.			
Attribute	Availability			
Environment	Normal operations			
Stimulus	Service X is a simple service; its provider is brought to a halt.			
Response	Failure of the service provider of service X is concealed from the user and service continuity is achieved within 5 seconds.			
Architectural decisions	sensitivity	Trade-off	Risk	Non-risk
Pattern r - Service migration and replication	S3	T4		
Service virtualization				
Reasoning	Transparent service migration – users are not aware of service migrations.			

Figure 4. Analysis of mobile SOA RA for scenario no. 6 of the utility tree.

instances. Figures 2–4 show an analysis of mobile SOA RA for three prioritized scenarios of the utility tree and sensitivity points, trade-off points and risks in these figures refer to Table V. Figures 2–4 follow the format presented by Bass, Clements, and Kazman [2003].

6.2. Evaluation of Possible Instances

Kazman et al. [2011] introduced an architecture-based analysis approach entitled "Architecture Evaluation without an Architecture." This approach to analysis applies foundational

Table V. Sensitivity Points, Trade-off Points and Risks

Type	Identifier	Description
Sensitivity point	S1	When the system prefetches an incorrect service, it wastes energy and memory on the consumer's device because it takes time to search the cache. Hence provision of sufficient statistical data is necessary for prefetching.
	S2	Encryption reduces the size of transmitting data but it requires processing time for compression/decompression and waiting time for both. So making determinations for the compression ratio and the compression algorithm is a sensitive point.
	S3	The number of service providers of a service has a direct relation with the availability of that service. Hence determining the number of service providers of a service is a sensitivity point.
Trade-off	T1	Prefetching can improve response time and availability but memory is consumed in the process of making a cache of prefetched services: in addition, cache invalidation uses up processing time and bandwidth.
	T2	A log of hit ratios of prefetching can be sent during a connection's idle time [Natchetoi, Kaufman, and Shapiro, 2008] or by piggybacking [Schreiber et al., 2011]; the latter makes a trade-off between energy consumption and response time [Schreiber et al., 2011].
	T3	Layering style (pattern) has processing overheads and increases response times because each message must be passed through layers and undergoes message conversions.
	T4	To achieve seamless service migration, the service provider must save checkpoints in another host; the frequency of these saving checkpoints creates a trade-off between availability, energy consumption and the bandwidth of the mobile service provider.
Risk	R1	The majority of the mobile SOA patterns concentrate on the constraints of a device's memory, bandwidth and processing power. Hence, integrating a system with cloud computing solutions cannot completely prevent ripple effects of removing these patterns.

principles of traditional architectural evaluation methods to analyze an architectural landscape: "a broad set of architectural decisions which represent a spectrum of potential architectures" [Kazman et al., 2011]. The approach serves to analyze an architecture landscape rather than a concrete architecture, this enables decisions to be made that can incorporate many stakeholders and integrate several systems presenting a complex and effective design [Kazman et al., 2011]. All possible instances of RA are examples of the landscapes enabling many possibilities. However, this can cause problems when making assessments for a design because effects of the compound effects of a multitude of these seemingly minor architecture decisions are compounded and long term effects may be difficult to determine accurately [Kazman et al., 2011]. Instantiating a mobile SOA from mobile SOA RA encompasses pattern-weaving and layering decisions whereas each pattern influences one or several quality attributes.

Instantiable architectures were evaluated by the method proposed by Kazman et al. [2011]. Briefly, this method uses business goals to develop scenarios that describe challenges to the system from multiple quality attributes, it develops an architectural landscape by considering the possible major alternatives for the creation and operation of the system and finally maps each scenario onto an architectural landscape to identify any risks and to determine assumptions behind each alternative.

The quality attributes defined in Section 3. were used to evaluate mobile SOA RA as business goals, although business goals of concrete mobile SOAs may include business qualities such as rollout schedules. Mobile SOA patterns (Section 4.) and mobile SOA RA (Section 5.) can be used to create an architectural landscape that incorporates all the possible alternatives available for application in a system. Each alternative represents an important architectural design criterion. The alternatives are based on choices for logical options such type of communication (push or pull communication); commercially available components such as types of networks, or design decisions within an architectural element such as frequency of communication [Kazman et al., 2011]. Fourteen architectural decisions (alternatives) of the landscape of mobile SOA RA instances are as follows. Each of these decisions must be considered and refined by the architect creating a specific mobile SOA.

- Transformations that are used (*pattern c*).
- Simplifications of SOAP that are used (*pattern c*).
- Which subset of the specifications of the SOAP and BPEL protocols can support planned services (*pattern g*).
- Decisions on numbers of consumers, providers and mediators and the bandwidth of fixed parts of the system's distribution of tasks (*pattern o*).
- If *pattern o* is selected, reliability of the mediator is important to avoid a single point of failure; using redundant servers for reliability creates a trade-off with cost and complexity.
- If *pattern x* is selected, the message format to start-up SOAP engine of a mobile host must be decided.
- If *pattern m* is selected, the protocol to transfer SOAP messages must be selected.

- If *pattern m* is selected, failure detection must be decided.
- If *pattern a* or *pattern b* is selected, the forecasting model of these patterns must be picked out.
- If connection idle time version of *pattern a* is selected, the frequency of prefetched messages must be decided.
- If *pattern a* is selected, the client side prefetching or the server side prefetching must be selected.
- If *pattern j* is selected, the connection management rules including preferences and thresholds for supported devices and applications must be decided.
- If connection merging of *pattern j* is selected, packet size must be decided.
- The metrics that the *Quality of Service* layer monitors must be chosen.

Scenarios are used to identify implications of challenges presented by architectural decisions. In contrast to ATAM, these scenarios address multiple quality attributes simultaneously. A set of scenarios is shown below:

- Mobile providers support a new network interface whereby some mobile devices have interfaces belonging to that network.
- Congestion of service messages affects the normal function of a mobile device.
- The mediator or ESB fails.
- The mobile device user simultaneously uses several applications so that constraints arise on resources of the mobile device. Consumption of resources due to prefetching (*pattern a*) and UI adaptation (*pattern b*) does not negatively affect normal functions of the mobile device.

Each scenario is mapped to architectural decisions of the landscape that might be used to achieve that scenario. This mapping relates assumptions to the landscape on which they have been built. However, some of these assumptions, or combinations of assumptions may serve to undermine the quality target of the system and undermine its effectiveness [Kazman et al., 2011].

For example analysis of the last scenario on the list above highlights the following risks:

- *Usability*: The consumer may be confused about changes that affect the UI.
- *Modifiability*: Changes in runtime elements of the mobile device create a trade-off with complexity.
- *Cost*: Changes in runtime elements of the mobile device create a trade-off with cost.
- *Performance*: allocating a considerable amount of resources (memory) for prefetching (*pattern a*) may negatively affect normal functioning of a mobile device but being parsimonious with the memory available for prefetching may reduce the time that a prefetched service resides in the memory of the mobile device, so that a service may be fetched to the cache (prefetched) and removed from the cache before use of the (cached) service takes place; furthermore, if the same service is subsequently invoked directly it will not be in the cache (removed). Hence considerable bandwidth, processing, memory and energy are wasted to repeat invocation of that prefetched service.

The first scenario poses the following risks:

- *Modifiability*: The network interface implemented by different vendors can have different levels of performance while these implemented network interfaces follow the same standards of interface. Hence connection management rules are necessary to prioritize network interfaces, these must be device specific, so adding new rules is expensive and time-consuming.
- *Usability*: Updating the connection manager component to support the new network interface may require a manual setup by the user.

6.3. An Overview of the Evaluations

Inspired from the adaptations proposed by Angelov, Trienekens, and Grefen [2008] and Gallagher [2000] the mobile SOA RA was evaluated by the ATAM for two sets of stakeholders of RA and this evaluation used both general and concrete scenarios. The utility tree and the analysis for the three prioritized scenarios of the utility tree and sensitivity points, trade-off points and risks pertained to these scenarios were presented.

To explore nuances of instantiation, an evaluation of possible instantiable architectures was carried out by the method proposed by Kazman et al. [2011]. This evaluation presented architectural alternatives as an architectural landscape and mapped scenarios that described challenges to the system from multiple quality attributes to this landscape. This mapping revealed some risks and assumptions behind architectural decisions, which alone or in combination pose potential risks to achievement of the targeted quality attributes.

7. CONCLUSION

This paper has proposed a mobile SOA RA, explored 26 mobile SOA patterns and investigated nuances of the concept of RA including an outline for a design procedure. Methods, configurations, and results of the both evaluations have been discussed. From these evaluations and extracted patterns it can be concluded that mobile SOAs have fixed trends or themes in their architectural decisions and the patterns associated with these trends can realize several of the abilities of Open Group SOA RA. The RA proposed in this paper is compatible with Open Group SOA RA [Open Group, 2011], which facilitates interoperability between mobile and non-mobile SOAs.

Mobile SOA RA provides a number of benefits for software architects. It allows faster development of mobile SOAs; it also facilitates interoperability between instantiated mobile SOAs and other SOAs which followed Open Group SOA RA [Open Group, 2011]. It introduces a standardized view of responsibilities in the development of mobile SOAs and provides the architect with patterns that constitute a toolbox of architectural knowledge. Moreover, the pattern-based approach of the proposed mobile SOA RA can facilitate

architecting of mobile SOAs in conjunction with incremental iterative development-methods.

Further research can facilitate instantiation in two ways—first, it is difficult for architects to incorporate a selection of patterns simultaneously [Ackerman and Gonzalez, 2010], hence relationships between the patterns are defined according to the following terms; extends, is part of, complements with and is applicable to [Stricker et al., 2010] These terms can then be used to facilitate weaving the pattern, and for a formal representation of variability [Nakagawa, 2012] by imposing restrictions to the set of variants which can be chosen together [Pohl, Böckle, and Linden, 2005] to facilitate instantiation. To the best of our knowledge, there are as yet no RA which provide these facilities, but it is supposed that sufficient instances are the key to achievement of both of these extensions. Furthermore, these extensions, along with the other mentioned benefits, can increase the number of rational incentives for wider adoption of mobile SOA RA.

REFERENCES

A. Acero, N. Bernstein, R. Chambers, Y.C. Ju, X. Li, J. Odell, P. Nguyen, O. Scholz, and G. Zweig, Live search for mobile: Web services by voice on the cellphone, IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2008, Las Vegas, NV, 2008, pp. 5256–5259.

L. Ackerman, and C. Gonzalez, Patterns-based engineering: Successfully delivering solutions via patterns, Addison-Wesley Professional, Boston, MA, USA, 2010.

A. Al-Hussain, and I. Al-Rassan, A biometric-based authentication system for web services mobile user, Proceedings of the 8th International Conference on Advances in Mobile Computing and Multimedia, MoMM '10, Paris, France, 2010, pp. 447–452.

M. Adaçal, and A.B. Bener, Mobile Web services: A new agent-based framework, Internet Comput 10(3) (2006), 58–65.

S. Angelov, J.J.M. Trienekens, and P. Grefen, Towards a method for the evaluation of reference architectures: Experiences from a case, Proceedings of the 2nd European conference on Software Architecture, Springer-Verlag, Paphos, Cyprus, 2008, pp. 225–240.

S. Angelov, P. Grefen, and D. Greefhorst, A framework for analysis and design of software reference architectures, Inform Software Technol 54(4) (2012), 417–431.

N. Apte, K. Deutsch, and R. Jain, Wireless SOAP: Optimizations for mobile wireless web services, Special Interest Tracks and Posters of the 14th International Conference on World Wide Web, WWW '05, 2005, pp. 1178–1179, http://dl.acm.org/citation.cfm?id=1062927.

A. Arsanjani, L.J. Zhang, M. Ellis, A. Allam, and K. Channabasavaiah, S3: A service-oriented reference architecture, IT Profess 9(3) (2007), 10–17.

M. Asif, S. Majumdar, and R. Dragnea, Partitioning the WS execution environment for hosting mobile web services, Proceedings of the 2008 IEEE International Conference on Services Computing, SCC '08, Vol. 2, Honolulu, HI, 2008, pp. 315–322.

M. Autili, M. Caporuscio, and V. Issarny, A reference model for service oriented middleware, Technical Report inria-00326479, INRIA Paris-Rocquencourt, 2008.

M. Autili, M. Caporuscio, and V. Issarny, Architecting service oriented middleware for pervasive networking, Proceedings of the 2009 ICSE Workshop on Principles of Engineering Service Oriented Systems, PESOS '09, Vancouver, BC, 2009, pp. 58–61.

F. Bachmann, L. Bass, and R. Nord, Understanding architectural patterns in terms of tactics and models, News at SEI, 1 August 2007, www.sei.cmu.edu/library/abstracts/news-at-sei/architect200708.cfm. (Last accessed Mar. 12, 2007).

L. Bass, P. Clements, and R. Kazman, Software architecture in practice, 2nd edition, Addison-Wesley Professional, Boston, MA, USA, 2003.

U. Bellur, and S. Bondre, Towards seamless user mobility in service oriented environments via context awareness, Proceedings of the 5th international conference on Pervasive services, ICPS '08, 2008, pp. 185–188.

G. Broll, J. Hamard, M. Paolucci, M. Haarlander, M. Wagner, S. Siorpaes, E. Rukzio, A. Schmidt, and K. Wiesner, Mobile interaction with web services through associated real world objects, Proceedings of the 9th international conference on Human computer interaction with mobile devices and services, MobileHCI '07, 2007, pp. 319–321.

G. Broll, M. Paolucci, M. Wagner, E. Rukzio, A. Schmidt, and H. Hußmann, Perci: Pervasive service interaction with the internet of things, Internet Comput 13(6) (2009), 1089–7801.

Y. Cao, and J. Li, An agent-based automatic services composition algorithm of mobile collaboration platform, Proceedings of the 2009 Second International Conference on Information and Computing Science, ICIC '09, vol. 2, Manchester, 2009, pp. 285–288.

M. Caporuscio, D. Charlet, and V. Issarny, Energetic performance of service-oriented multi-radio networks: Issues and perspectives, Proceedings of the 6th international workshop on Software and performance, WOSP '07, 2007, pp. 42–45.

M. Caporuscio, P.G. Raverdy, and V. Issarny, ubiSOAP: A service-oriented middleware for ubiquitous networking, IEEE Trans Services Comput 5(1) (2012), 86–98.

C. Chang, S. Ling, and S. Krishnaswamy, ProMWS: Proactive mobile web service provision using context-awareness, 2011 IEEE International Conference on Pervasive Computing and Communications Workshops (PERCOM Workshops), Seattle, WA, 2011, pp. 69–74.

C.C. Chang, J.C.R. Tseng, and K.J. Lin, A dynamic capability framework for context-aware mobile services, Proceedings of the 2008 10th IEEE Conference on E-Commerce Technology and the Fifth IEEE Conference on Enterprise Computing, E-Commerce and E-Services, CECANDEEE '08, Washington, DC, 2008, pp. 183–189.

C.E. Chang, F. Mohd-Yasin, and A.K. Mustapha, An implementation of embedded RESTful web services, Innovative Technologies in Intelligent Systems and Industrial Applications, CITISIA 2009, Monash, 2009, pp. 45–50.

R. Cloutier, G. Muller, D. Verma, R. Nilchiani, E. Hole, and M. Bone, The concept of reference architectures, Syst Eng 13(1) (2010), 14–27.

R.J. Cloutier, D. Verma, Applying the concept of patterns to systems architecture, Syst Eng 10(2) (2007), 138–154.

K. Chujo, Y. Ota, K. Takaki, and S. Shiitani, Human centric engine and its evolution toward web services, FUJITSU Scient Technol J 49(2) (2013), 153–159.

I. Duda, M. Aleksy, and T. Butter, Architectures for mobile device integration into service-oriented architectures, Proceedings of the International Conference on Mobile Business, ICMB '05, 2005, pp. 193–198.

H.J. Eikerling, M. Benesch, and F. Berger, Using proximity relations for the adaptation of mobile field services, International Workshop on Engineering of Software Services for Pervasive Environments: In Conjunction with the 6th ESEC/FSE Joint Meeting, ESSPE '07, 2007, pp. 53–57.

K. Elgazzar, P. Martin, and H. Hassanein, "A framework for efficient web services provisioning in mobile environments", in J.Y. Zhang, J. Wilkiewicz, A. Nahapetian (Editors), Mobile Computing, Applications, and Services, Vol. 95, Springer, Los Angeles, CA, 2012, pp. 246–262.

T. Erl, SOA design patterns, Prentice Hall, Boston, MA, USA, 2008.

B.P. Gallagher, Using the architecture tradeoff analysis method to evaluate a reference architecture: A case study, Technical Note CMU/SEI-2000-TN-007, Software Engineering Institute, Carnegie Mellon University, 2000, http://www.sei.cmu.edu/library/abstracts/reports/00tn007.cfm. (Last accessed Mar. 12, 2012).

Gartner, Gartner identifies the top 10 Strategic Technologies for 2011, 19 October 2010, http://www.gartner.com/it/page.jsp?id=1454221. (Last accessed Mar. 23, 2013).

Gartner, Gartner Identifies the Top 10 Strategic Technologies for 2012, 18 October 2011, http://www.gartner.com/it/page.jsp?id=1826214. (Last accessed Mar. 23, 2013).

Gartner, Gartner Identifies the Top 10 Strategic Technology Trends for 2013, 23 October 2012, http://www.gartner.com/newsroom/id/2209615. (Last accessed Mar. 23, 2013).

D. Griol, N. Sanchez-Pi, J. Carbo, and J.M. Molina, Context-aware approach for orally accessible web services, Proceedings of the 2009 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology, WI-IAT '09, vol. 3, Milan, Italy, 2009, pp. 171–174.

J.V. Gurp, A. Karhinen, and J. Bosch, Mobile service oriented architectures (MOSOA), Proceedings of the 6th IFIP WG 6.1 international conference on Distributed Applications and Interoperable Systems, DAIS'06, Springer-Verlag, 2006, pp. 1–15.

L. Hamdi, H. Wu, and S. Dagtas, Ajax for mobility: MobileWeaver ajax framework, Proceedings of the 17th international conference on World Wide Web, WWW '08, 2008, pp. 1077–1078.

C. Haskins, Using patterns to transition systems engineering from a technological to social context, Syst Eng 11(2) (2008), 147–155.

N. Henze, E. Rukzio, A. Lorenz, X. Righetti, and S. Boll, Physical-virtual linkage with contextual bookmarks, Proceedings of the 10th International Conference on Human Computer Interaction with Mobile Devices and Services, MobileHCI '08, 2008, pp. 523–526.

S. Hoober, and E. Berkman, Designing mobile interfaces, O'Reilly Media, Sebastopol, CA, 2011.

IBM, The flexible workplace: Unlocking value in the "bring your own device" era, IBM Global Technology Services Thought Leadership White Paper, November 2012, http://www.ibm.com/midmarket/it/it/att/pdf/1211˙The˙flexible˙workplace˙BYOD.pdf. (Last accessed Mar. 23, 2013).

ISO/IEC/IEEE Std. 42010:2011(E), Systems and software engineering—Architecture description, Switzerland, 2011.

I. Jørstad, D.V. Thanh, and S. Dustdar, Personalisation of Future Mobile Services, 2004, http://www.infosys.tuwien.ac.at/Staff/sd/papers/ICIN2004-PersonalisationOfFutureMobileServices.pdf. (Last accessed Mar. 12, 2012).

R. Kazman, L. Bass, J. Ivers, and G.A. Moreno, Architecture evaluation without an architecture: Experience with the smart grid, Proceedings of the 33rd International Conference on Software Engineering, ICSE '11, Honolulu, HI, 2011, pp. 663–670.

Y.S. Kim, and K.H. Lee, A lightweight framework for mobile web services, Comput Sci – Res Dev 24(4) (2009), 199–209.

J.Y. Lee, H.J. Lee, D.V.T. Tran, H.J. La, and S.D. Kim, Dynamic deployment of services for service-based mobile ecosystem, 2011 IEEE 8th International Conference on e-Business Engineering, Beijing, China, 2011, pp. 229–236.

P. Leong, C. Miao, B.K. Lim, J.W. Lim, C. Chen, N. Dianna, and W.B. Toh, Agent mediated peer-to-peer mobile service-oriented architecture, Inaugural IEEE-IES Digital EcoSystems and Technologies Conference, DEST 07, Cairns, 2007, pp. 414–419.

G. Li, and H. Sun, RESTful dynamic framework for services in mobile wireless networks, International Conference on E-Business and Information System Security, EBISS '09, Wuhan, 2009, pp. 1–5.

X. Liu, and R. Deters, An efficient dual caching strategy for web service-enabled PDAs, Proceedings of the 2007 ACM Symposium on Applied Computing, SAC '07, 2007, pp. 788–794.

Y. Liu, and K. Connelly, Realizing an open ubiquitous environment in a RESTful Way, Proceedings of the 2008 IEEE International Conference on Web Services, ICWS '08, Beijing, 2008, pp. 96–103.

Y. Liu, and F. Li, PCA: A reference architecture for pervasive computing, 2006 1st International Symposium on Pervasive Computing and Applications, Urumqi, 2006, pp. 99–103.

S. Marzouk, A.J. Maalej, I.B. Rodriguez, and M. Jmaıel, Periodic checkpointing for strong mobility of orchestrated web services, Proceedings of the 2009 Congress on Services—I, SERVICES '09, Los Angeles, CA, 2009, pp. 203–210.

G. Muller, and P.V.D. Laar, "Researching Reference Architectures", in P.V.D. Laar and T. Punter (Editors), Views on evolvability of embedded systems, Springer, 2011, pp. 107–119, http://link.springer.com/chapter/10.1007/978-90-481-9849-8_7.

E.Y. Nakagawa, Reference architectures and variability: Current status and future perspectives, Proceedings of the WICSA/ECSA 2012 Companion Volume, 2012, pp. 159–162.

E.Y. Nakagawa, M. Becker, and J.C. Maldonado, A knowledge-based framework for reference architectures, Proceedings of the 27th Annual ACM Symposium on Applied Computing, 2012, pp. 1197–1202.

Y. Natchetoi, V. Kaufman, and A. Shapiro, Service-oriented architecture for mobile applications, Proceedings of the 1st international workshop on Software architectures and mobility, SAM '08, 2008, pp. 27–32.

Q. Ni, and M. Sloman, An Ontology-enabled service oriented architecture for pervasive computing, Proceedings of the International Conference on Information Technology: Coding and Computing, ITCC '05, vol. 2, 2005, pp. 797–798.

S. Oh, and G.C. Fox, Optimizing Web Service messaging performance in mobile computing, Future Generat Comput Syst 23(4) (2007), 623–632.

L.B.R. Oliveira, K.R. Felizardo, D. Feitosa, and E.Y. Nakagawa, Reference models and reference architectures based on service-oriented architecture: A systematic review, Proceedings of the 4th European conference on Software architecture, ECSA'10, 2010, pp. 360–367.

Open Group, SOA Reference Architecture, Draft Technical Standard, Draft 10, 2009, http://www.opengroup.org/projects/soa-ref-arch/uploads/40/19713/soa-ra-public-050609.pdf. (Last accessed Mar. 12, 2012).

Open Group, SOA Reference Architecture, Technical Standard, Document Number C119, 2011, https://www2.opengroup.org/ogsys/jsp/publications/PublicationDetails.jsp?catalogno=c119. (Last accessed Mar. 12, 2012).

Z. Ou, M. Song, H. Chen, and J. Song, Layered peer-to-peer architecture for mobile web services via converged cellular and ad hoc networks, Proceedings of the 2008 The 3rd International Conference on Grid and Pervasive Computing—Workshops, GPC-WORKSHOPS '08, Kunming, 2008, pp. 195–200.

A. Papageorgiou, J. Blendin, A. Miede, J. Eckert, and R. Steinmetz, Study and comparison of adaptation mechanisms for performance enhancements of mobile web service consumption, Proceedings of the 2010 6th World Congress on Services, SERVICES '10, Miami, FL, 2010, pp. 667–670.

A. Papageorgiou, U. Lampe, D. Schuller, and R. Steinmetz, Invoking web services based on energy consumption models, IEEE First International Conference on Mobile Services (MS 2012), 2012a, pp. 40–47.

A. Papageorgiou, M. Schatke, S. Schulte, and R. Steinmetz, Lightweight wireless web service communication through enhanced caching mechanisms, Int J Web Services Res 9(2) (2012b), 42–68.

P.V. Paul, D. Rajaguru, N. Saravanan, R. Baskaran, and P. Dhavachelvan, Efficient service cache management in mobile P2P networks, Future Generation Computer Systems (2013).

P. Pawar, S. Srirama, B.J.V. Beijnum, and A.V. Halteren, A comparative study of nomadic mobile service provisioning approaches, Proceedings of the 2007 International Conference on Next Generation Mobile Applications, Services and Technologies, NGMAST '07, Cardiff, 2007, pp. 277–286.

F. Pfister, V. Chapurlat, M. Huchard, C. Nebut, and J.-L. Wippler, A proposed meta-model for formalizing systems engineering knowledge, based on functional architectural patterns, Syst Eng 15(3) (2012), 321–332.

E. Philips, R.V.D. Straeten, and V. Jonckers, NOW: Orchestrating services in a nomadic network using a dedicated workflow language, Sci Comput Program 78(2) (2013), 168–194.

P. Plebani, C. Cappiello, M. Comuzzi, B. Pernici, and S. Yadav, MicroMAIS: Executing and orchestrating web services on constrained mobile devices, Software: Pract Exp 42(9) (2012), 1075–1094.

K. Pohl, G. Böckle, and F.V.D. Linden, Software product line engineering: Foundations, principles, and techniques, Springer-Verlag, Berlin, 2005.

L.D. Prete, and L. Capra, MoSCA: Service Composition in Mobile Environments, Proceedings of the ACM/IFIP/USENIX Middleware '08 Conference Companion, Companion '08, 2008, pp. 87–89.

P. Reed, Reference architecture: The best of best practices, IBM developer works, 15 September 2002, http://www.ibm.com/developerworks/rational/library/2774.html. (Last accessed Mar. 12, 2012).

A. Reyes, R. Messeguer, D. Royo, and C. Homar, Mobile access to the services in Ambient Networks, 4th International Conference on Intelligent Environments, Seattle, Washington, 2008, pp. 1–4.

A. Rodriguez, RESTful Web services: The basics, IBM developer works, 06 November 2008, http://www.ibm.com/developerworks/webservices/library/ws-restful. (Last accessed Mar. 12, 2012).

H. Schmidt, R. Kapitza, F.J. Hauck, and H.P. Reiser, AWSM: Infrastructure for adaptive web service migration, Proceedings of the 2007 OTM confederated international conference on On the move to meaningful internet systems, OTM'07, vol. 1, 2007, pp. 3–4.

D. Schreiber, A. Göb, E. Aitenbichler, and M. Mühlhäuser, Reducing user perceived latency with a proactive prefetching middleware for mobile SOA access, Int J Web Ser Res 8(1) (2011), 68–85.

B. Sefid-Dashti, and J. Habibi, A conceptual usability framework for mobile service consumers, Int J Comput Technol Appl 2(4) (2011), 894–903.

A. Shajjar, N. Khalid, H.F. Ahmad, and H. Suguri, Service interoperability between agents and semantic web services for nomadic environment, Proceedings of the 2008 IEEE/WIC/ACM International Conference on Web Intelligence and Intelligent Agent Technology, WI-IAT'08, vol. 2, Sydney, NSW, 2008, pp. 583–586.

Q.Z. Sheng, Z. Maamar, J. Yu, and A.H.H. Ngu, Robust web services provisioning through on-demand replication, Proceedings of Third International United Information Systems Conference, UNISCON 2009, Sydney, Australia, 2009, pp. 4–16.

R. Singh, S. Mishra, and D.S. Kushwaha, An efficient asynchronous mobile web service framework, Newslett: ACM SIGSOFT Software Eng Notes 34(6) (2009), 1–7.

S.N. Srirama, M. Jarke, and W. Prinz, A Mediation Framework for Mobile Web Service Provisioning, Proceedings of the 10th IEEE on International Enterprise Distributed Object Computing Conference Workshops, EDOCW '06, Hong Kong, China, 2006.

S.N. Srirama, M. Jarke, and W. Prinz, Mobile web services mediation framework, Proceedings of the 2nd workshop on Middleware for service oriented computing: held at the ACM/IFIP/USENIX International Middleware Conference, MW4SOC '07, 2007, pp. 6–11.

V. Stricker, K. Lauenroth, P. Corte, F. Gittler, S. D. Panfilis, and K. Pohl, "Creating a reference architecture for service-based systems—A pattern-based approach", in G. Tselentis, A. Galis, A. Gavras, S. Krco, V. Lotz, E. Simperl, B. Stiller, and T. Zahariadis (Editors), Towards the future internet—Emerging trends from European research, IOS Press, 2010, pp. 149–160, http://ebooks.iospress.nl/publication/30728.

A.S. Tanenbaum, and M.V. Steen, Distributed systems principles and paradigms, 2nd edition, Prentice Hall, Upper Saddle River, NJ, 2006.

R.N. Taylor, N. Medvidovic, and E.M. Dashofy, Software architecture: Foundations, theory, and practice, John Wiley & Sons, USA, 2009.

D.V. Thanh, and I. Jørstad, A service-oriented architecture framework for mobile services, Proceedings of the Advanced Industrial Conference on Telecommunications/Service Assurance with Partial and Intermittent Resources Conference/E-Learning on Telecommunications Workshop, AICT-SAPIR-ELETE '05, 2005, pp. 65–70.

M. Tian, T. Voigt, T. Naumowicz, H. Ritter, and J. Schiller, Performance considerations for mobile web services, Comput Commun 27 (11) (2004), 1097–1105.

H.L. Truong, and S. Dustdar, A survey on context-aware web service systems, Int J Web Inform Syst 5(1) (2009) 5–31.

V.S. Tseng, and K.W. Lin, Efficient mining and prediction of user behavior patterns in mobile web systems, Inform Software Technol 48(6) (2006), 357–369.

S. Vinoski, REST Eye for the SOA Guy, IEEE Internet Comput 11(1) (2007), 82–84.

H. Wu, and Y. Natchetoi, Mobile shopping assistant: Integration of mobile applications and web services, Proceedings of the 16th international conference on World Wide Web, WWW '07, 2007, pp. 1259–1260.

W. Zahreddine, and Q.H. Mahmoud, An agent-based approach to composite mobile Web services, Proceedings of the 19th International Conference on Advanced Information Networking and Applications, AINA '05, vol. 2, 2005, pp. 189–192.

L.J. Zhang, and J. Zhang, "SOA reference architecture", in L.J. Zhang (Editor), Web services research for emerging applications: Discoveries and trends, Information Science Reference, 2010, pp 1–15.

Behrouz Sefid-Dashti received his B.S. and M.S. degrees in computer engineering from the Iran Information Technology Development and Islamic Azad University (Tehran North Branch), respectively. His career and experience include software analysis, business analysis, software architecture, and business architecture. His research interests focus on software architecture, enterprise architecture, and mobile computing.

Jafar Habibi received a faculty member of the Computer Engineering department at Sharif University of Technology and is the managing director of Machine Services. He is the supervisor of Sharif's Robo-Cup Simulation Group. He received his B.S. degree in computer engineering from the Supreme School of Computer, his M.S. degree in industrial engineering from Tarbiat Modares Univers ity and his Ph.D. degree in computer engineering from Manchester University. His research interests are in computer engineering, simulation systems, MIS, DSS, and evaluation of computer systems performance.